

A Latency Handbook for Software Engineers



Quang Hoang

A Latency Handbook For Software Engineers

Bạn có biết Amazon từng tính toán rằng cứ mỗi 100ms latency tăng thêm, họ mất 1% doanh thu? Hay Google phát hiện ra rằng tăng latency thêm 500ms sẽ làm giảm traffic 20%?

Trong System Design, làm cho hệ thống chạy "đúng" (correctness) là điều kiện cần, nhưng làm cho nó chạy "nhanh" (low latency) mới là điều kiện đủ để hệ thống sống sót ở quy mô lớn.

Cuốn ebook ngắn này tổng hợp lại những “kiến thức, kinh nghiệm, và trải nghiệm” của bản thân mình về Latency trong hệ thống phần mềm.

Mục lục

- [1. Định nghĩa Latency](#)
 - [1.1. Định nghĩa](#)
 - [1.2. Latency và Throughput](#)
- [2. Đo Latency như thế nào?](#)
 - [2.1. Mean latency](#)
 - [2.2. Percentile \(P50, P90, P99\)](#)
- [3. Nguyên nhân của High Latency](#)
 - [3.1. Propagation](#)
 - [3.1.1. Những cái bắt tay](#)
 - [3.1.2. Packet Loss và Retransmission](#)
 - [3.2. Queuing](#)
 - [3.2.1. Little's Law](#)
 - [3.2.2. Utilization Law](#)
 - [3.3. Processing delay](#)
 - [3.3.1. Garbage Collection \(GC\)](#)
 - [3.3.2. I/O Wait](#)
- [4. The Tactics](#)
 - [4.1. Avoid data movement](#)
 - [4.1.1. Collocation](#)
 - [4.1.2. Replication](#)
 - [4.1.3. Local Caching](#)
 - [4.2. Avoid Work](#)
 - [4.2.1. Database Index](#)
 - [4.2.2. Optimize SQL Query](#)
 - [4.2.3. Optimize Code Logic](#)
 - [4.2.4. Remote caching](#)
 - [4.2.5. Thread Pool/ Connection Pool](#)
 - [4.3. Avoid Waiting](#)
 - [4.3.1. Event Loop & I/O Multiplexing](#)
 - [4.3.2. Minimize Synchronization](#)
 - [4.3.3. Partitioning / Sharding](#)
 - [4.4. Hide Latency](#)
 - [4.4.1. Async Processing](#)
 - [4.4.2. Parallelism](#)
 - [4.4.3. Request Hedging](#)
 - [4.4.4. Request Collapsing \(Singleflight\)](#)
 - [4.4.5. Speculative Prefetching](#)

[Lời Kết](#)

[Tài liệu tham khảo thêm](#)

[Về tác giả](#)

1. Định nghĩa Latency

1.1. Định nghĩa

Đa số mọi người nghĩ Latency đơn giản là: "Thời gian từ lúc gửi request đến lúc nhận response". Đúng, nhưng chưa đủ.

Trong System Design, Latency nên được nhìn dưới góc độ của **Flow**. Latency là tổng thời gian một đơn vị dữ liệu (packet, request, task) dành ra để đi qua hệ thống của bạn.

Nó bao gồm thời gian di chuyển (travel), thời gian chờ đợi (wait), và thời gian được xử lý (service).

$$Latency = Propagation + Queueing + Service$$

1.2. Latency và Throughput

"Nhanh" là một từ mơ hồ. Khi sếp bảo bạn "làm cho hệ thống nhanh lên", sếp đang muốn giảm Latency hay tăng Throughput? Hai mục tiêu này đôi khi mâu thuẫn nhau. Hãy tưởng tượng bạn được giao nhiệm vụ vận chuyển hành khách từ Hà Nội đến Hải Phòng (khoảng 100km).

Chiếc xe đua F1 (Đại diện cho Low Latency)

Bạn có một chiếc xe F1. Nó chạy với vận tốc 300km/h.

- **Sức chứa:** 1 người.
- **Thời gian di chuyển:** 20 phút.

Nếu bạn là một VIP cần đến Hải Phòng gấp, chiếc F1 là lựa chọn hoàn hảo. **Latency rất thấp (20 phút).**

Chiếc xe Bus (Đại diện cho High Throughput)

Bạn có một chiếc xe bus cũ chỉ chạy được 50km/h.

- **Sức chứa:** 50 người.
- **Thời gian di chuyển:** 2 tiếng (120 phút).

Nếu xét về độ trễ, xe bus thua xa xe đua (chậm hơn 6 lần). Nhưng giả sử bạn cần đưa **1.000 người** từ Hà Nội xuống Hải Phòng.

Cách 1: Dùng xe F1

Xe F1 phải chạy đi chạy lại 1.000 chuyến (giả sử thời gian quay đầu bằng 0 cho đơn giản).

- Latency trung bình của 1 người: 20 phút (Rất nhanh).
- Throughput: 1 người / 20 phút = **3 người/giờ**.
- Để chờ hết 1.000 người, bạn mất **333 giờ (gần 2 tuần)**

Cách 2: Dùng đoàn xe Bus

Bạn xuất phát 20 chiếc xe bus cùng lúc.

- Latency trung bình của 1 người: 120 phút (Chậm).
- Throughput: 50 người * 20 xe / 120 phút = **500 người/giờ**.
- Để chờ hết 1.000 người, bạn chỉ mất **2 giờ**.

Trong thực tế, rất khó để tối ưu cả Latency lẫn Throughput. Tùy thuộc vào bản chất của hệ thống và kỳ vọng của người dùng mà bạn có thể phải đánh đổi 1 trong 2 chỉ số này.

2. Đo Latency như thế nào?

2.1. Mean latency

Giả sử hệ thống của bạn xử lý 10 requests:

- 9 requests mất 1ms.
- 1 request mất 1000ms (do Garbage Collection hoặc nghẽn mạng).

Giá trị trung bình (Average): $(9 * 1 + 1000) / 10 \sim 100\text{ms}$.

Nhìn vào con số này, bạn thấy cũng ổn. Nhưng thực tế, có 1% user đang phải chờ tới 1 giây! Average che giấu những điểm bất thường (outliers).

✅ **Bài học:** Đừng bao giờ nhìn vào Average Latency để đánh giá hiệu suất hệ thống.

2.2. Percentile (P50, P90, P99)

Thay vì Average Latency, hãy dùng Percentile.

- **P50 (Median):** 50% số requests nhanh hơn mức này.
- **P90:** 90% số requests nhanh hơn mức này. Có nghĩa là cứ 100 requests thì có 10 request chậm hơn mức này.

- **P99**: 99% số requests nhanh hơn mức này. Có nghĩa là cứ 100 requests thì có 1 request chậm hơn mức này.

Tại sao **P99** quan trọng? Ở quy mô lớn, "1% bị chậm" là con số khổng lồ. Hãy tưởng tượng trang chủ Amazon cần gọi tới 100 micro-services bên dưới. Nếu mỗi service có tỉ lệ lỗi/chậm là 1% (P99), xác suất để user load xong trang chủ mượt mà là:

$$0.99^{100} \approx 36.6\%$$

Nghĩa là **63%** user sẽ gặp vấn đề!

3. Nguyên nhân của High Latency

Như đã nói ở chương 1, Latency bao gồm thời gian di chuyển (travel), thời gian chờ đợi (wait), và thời gian được xử lý (service)

$$Latency = Propagation + Queueing + Service$$

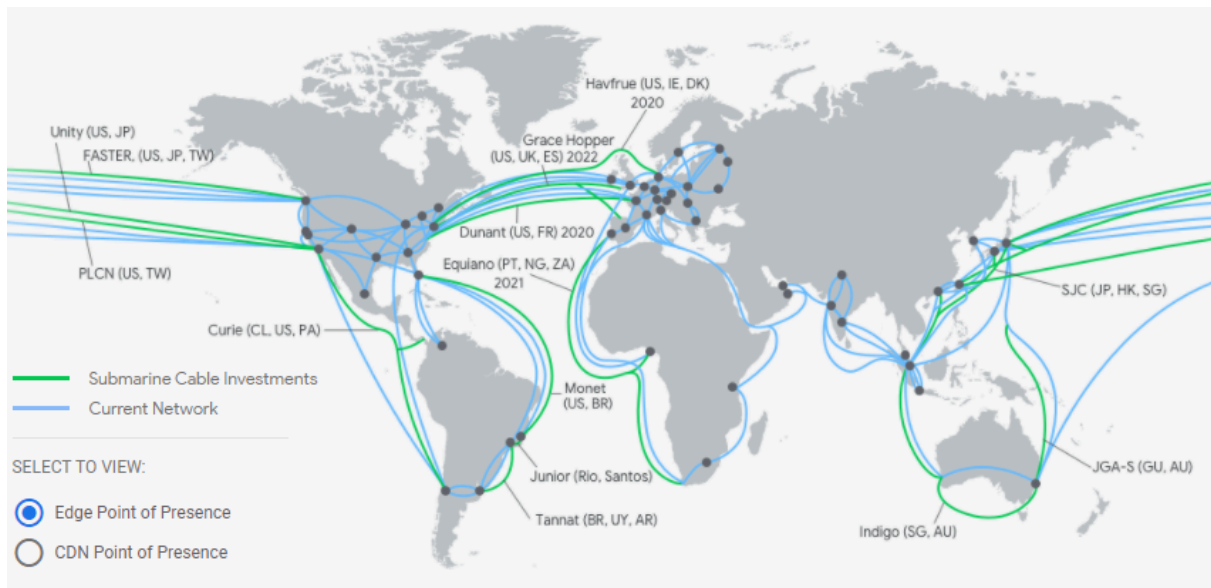
Hãy cùng đào sâu vào mỗi thành tố trong chương này.

3.1. Propagation

Đây là yếu tố đầu tiên và cũng là yếu tố khó chịu nhất vì nó bị giới hạn bởi vật lý. Trong chân không, ánh sáng đi được ~300.000 km/s. Trong cáp quang (fiber), tốc độ này giảm xuống còn khoảng 2/3, tức là ~200.000 km/s. Một gói tin (packet) đi từ New York sang London và quay về (Round Trip Time - RTT) sẽ mất tối thiểu ~50ms.

Nếu App của bạn đặt Database ở Mỹ nhưng phục vụ User ở Việt Nam, một câu query đơn giản nhất cũng mất **200-300ms** chỉ để dữ liệu chạy đi chạy về. Không một dòng code tối ưu nào có thể thắng được tốc độ ánh sáng.

Đó là lý do các công ty lớn thường đầu tư đặt data center ở khắp thế giới, việc này giúp giảm khoảng cách vật lý giữa user và server. Dưới đây là hình ảnh hệ thống data center và đường cáp mạng nội bộ của Google Cloud.



3.1.1. Những cái bắt tay

Trước khi gửi dữ liệu thật, các server còn phải làm thủ tục "chào hỏi".

1. **TCP Handshake (SYN - SYN/ACK - ACK):** Mất 1 RTT (Round Trip Time).
2. **TLS Handshake (HTTPS):** Để bảo mật, cần thêm 2 RTT nữa để trao đổi key mã hóa.

Tức là trước khi user nhìn thấy byte dữ liệu đầu tiên, họ đã mất **3 RTT** (khoảng vài trăm ms) chỉ để hai máy tính chào nhau. Đây là lý do tại sao các giao thức mới như **QUIC (HTTP/3)** ra đời để giảm số vòng "chào hỏi" này xuống.

3.1.2. Packet Loss và Retransmission

Mạng internet là một hệ thống không hoàn hảo. Đôi khi, gói tin bị "rớt" (packet loss) do router bị quá tải hoặc cá mập cắn cáp. Khi rớt gói tin, giao thức TCP sẽ phải gửi lại (retransmit). Việc này làm tăng latency đột biến. Một request bình thường mất 50ms, nhưng nếu rớt gói tin, nó có thể vọt lên 300ms hoặc hơn.

3.2. Queuing

Đây là nơi nhiều hệ thống "chết" khi scale (die at scale).

Hãy tưởng tượng server của bạn là một nhân viên thu ngân.

- **Processing Time:** Thời gian quét mã vạch và tính tiền (ví dụ: 1 phút).
- **Queuing Time:** Thời gian bạn đứng xếp hàng chờ đến lượt.

Nếu siêu thị vắng, bạn được phục vụ ngay. Latency = **1 phút**. Nhưng vào giờ cao điểm, có 10 người đứng trước bạn. Dù thu ngân vẫn làm việc nhanh như cũ (1 phút/người), bạn phải chờ 10 phút mới được phục vụ. Latency lúc này là **11 phút**!

Trong máy tính, hàng đợi (Queue) xuất hiện ở khắp nơi:

- Hàng đợi ở Router mạng.
- Hàng đợi vào CPU (OS Run Queue).
- Hàng đợi Connection Pool của Database.
- Hàng đợi Thread Pool của Web Server.

3.2.1. Little's Law

Định luật Little cho ta thấy mối liên hệ giữa Latency và Throughput

$$L = \lambda \times W$$

- **L (Load)**: Số lượng request đang nằm trong hệ thống (đang chờ + đang xử lý)
- **λ (Throughput)**: Tốc độ request đi vào (requests/sec).
- **W (Wait time/Latency)**: Thời gian trung bình một request ở trong hệ thống.

Hãy nhìn công thức biến đổi: $W = L/\lambda$. Nếu hệ thống của bạn (ví dụ Database) chỉ có khả năng xử lý song song giới hạn (ví dụ max 50 connections), thì khi lượng request λ tăng lên đột biến, Latency W bắt buộc phải tăng. Không có cách nào khác!

Khi hàng đợi đầy (L đạt max), hệ thống sẽ chuyển sang trạng thái bão hòa. Request mới đến sẽ bị reject hoặc bị timeout.

✅ **Bài học**: Queuing Delay thường chiếm tỉ trọng nhỏ khi traffic thấp, nhưng nó sẽ tăng theo hàm mũ khi traffic tiệm cận giới hạn chịu đựng của hệ thống.

3.2.2. Utilization Law

Trong lý thuyết hàng đợi (cụ thể là mô hình M/M/1 - mô hình sát thực tế với máy tính), mối quan hệ giữa độ bận (Utilization) và Latency được mô tả bằng công thức:

$$R = \frac{S}{1 - U}$$

Trong đó:

- **R (Response Time/Latency):** Tổng thời gian User phải chờ.
- **S (Service Time):** Thời gian CPU thực sự xử lý (ví dụ: mất 10ms để code chạy xong logic).
- **U:** Độ bận của CPU (từ 0% đến 100%).

Giả sử code của bạn chạy mất $S = 100ms$.

- Khi CPU bận 50% ($U=0.5$)

$$R = 100 / (1 - 0.5) = 200ms$$

Latency tăng gấp đôi so với lý tưởng. Chấp nhận được.

- Khi CPU bận 80% ($U=0.8$):

$$R = 100 / (1 - 0.8) = 500ms$$

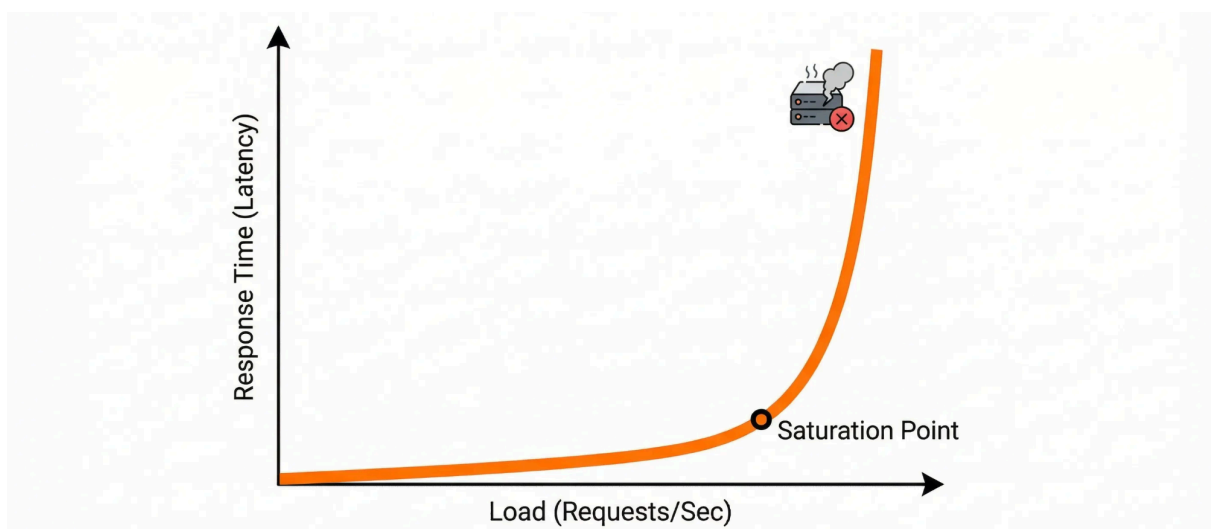
Latency tăng gấp 5 lần. Bắt đầu thấy chậm.

- Khi CPU bận 95% ($U=0.95$):

$$R = 100 / (1 - 0.95) = 2000ms$$

Latency tăng gấp **20** lần!

Đồ thị biểu diễn mối quan hệ này có hình dáng giống như một cây gậy khúc côn cầu (**Hockey Stick Curve**): Nó đi ngang rất lâu rồi đột ngột dựng đứng lên.



Lý do nằm ở sự **Ngẫu nhiên (Randomness)**. Trong thế giới thực, request của người dùng không đến đều đặn như nhịp tim (cứ đúng 100ms có 1 người bấm chuột) mà

chúng tuân theo **Phân phối Poisson**:

- Có giây không có user nào.
- Có giây rất nhiều user truy cập cùng lúc.

Khi server chạy ở mức 90% CPU, nó không có **khoảng hở (headroom)** để xử lý những sự gia tăng đột ngột này. Trong thực tế, chúng ta cần chạy load test để xác định điểm bão hoà (**Saturation Point**) của Utilization. Đây là điểm mà latency bắt đầu tăng nhanh.

Ví dụ nếu Saturation point = 70%~80%:

- **Vùng an toàn (< 70%)**: Latency ổn định.
- **Vùng nguy hiểm (70% - 80%)**: Latency bắt đầu tăng nhanh.
- **Vùng chết (> 80%)**: Latency tăng theo hàm mũ. Hệ thống coi như "treo".

✓ **Bài học**: luôn giữ Utilization của server ở một khoảng hờ dưới mức bão hoà.

3.3. Processing delay

Cuối cùng, sau khi request vượt qua được network và xếp hàng chờ đợi (queue), nó mới thực sự được server xử lý. Đây thường là phần dễ tối ưu nhất vì bạn có thể trực tiếp can thiệp bằng code.

Nhưng Processing Delay không chỉ là "thời gian chạy thuật toán". Nó bao gồm cả những thứ "vô hình" mà chúng ta hay bỏ quên.

3.3.1. Garbage Collection (GC)

Với các ngôn ngữ như Java, Go, Python, việc dọn dẹp bộ nhớ là tự động. GC là 1 sự kiện Stop-the-world, kiểu như "Tất cả đứng im, không ai được động dậy để tôi quét nhà". Trong lúc GC chạy, toàn bộ application của bạn bị đóng băng. Không request nào được xử lý.

- Nếu GC chạy nhanh (vài ms): User không cảm thấy gì.
- Nếu Memory bị leak hoặc phân mảnh quá nhiều, GC có thể chạy mất hàng trăm ms, thậm chí vài giây.

Đây là 1 trong những lý do khiến Discord chuyển từ Cassandra qua ScyllaDB: Cassandra (chạy trên JVM) gặp vấn đề với GC, dẫn đến các đợt giật lag (spike) độ trễ, có khi tăng đến 10s, gây ra sự không ổn định. ScyllaDB được viết bằng C++, không sử dụng GC, giúp hiệu năng ổn định và nhanh hơn. Các bạn có thể đọc thêm [tại đây](#).

3.3.2. I/O Wait

Thực ra, trong phần lớn thời gian gọi là "Processing", CPU chẳng làm gì cả. Nó chỉ ngồi đợi dữ liệu từ ổ cứng (Disk I/O) hoặc đợi kết quả từ server khác trả về (Network I/O).

Tóm lại để giảm Latency, bạn phải bóc tách nguyên nhân cụ thể:

- Nếu chậm do **Network**: Hãy mang data lại gần user (CDN, Edge Computing).
- Nếu chậm do **Queuing**: Hãy scale out (thêm server) hoặc giảm traffic (rate limiting). Hãy nhớ Little's Law.
- Nếu chậm do **Processing**: Hãy optimize code, xem lại thuật toán, tune lại Garbage Collector hoặc Database Index.

Trong chương tiếp theo, chúng ta sẽ đi vào cụ thể từng chiến thuật để tối ưu Latency.

4. The Tactics

Trong chương này, chúng ta sẽ đi vào cụ thể từng chiến thuật để tối ưu Latency. Nguyên tắc tổng quát là

Cách nhanh nhất để làm một việc là đừng làm việc đó.

4.1. Avoid data movement

Dữ liệu càng ở gần nơi xử lý thì càng nhanh. Bạn cần nhớ bảng này (những ước lượng này được chia sẻ bởi Jeff Dean vào năm 2011 tại Stanford, [nguồn](#))

Hoạt động	Latency
Truy cập bộ nhớ đệm L1 (L1 cache reference)	0.5 ns
Truy cập bộ nhớ đệm L2 (L2 cache reference)	3 ns
Dự đoán sai nhánh CPU (Branch mispredict)	5 ns
Mutex lock/unlock	15 ns
Truy cập RAM	50 ns
Đọc 4KB từ SSD	20,000 ns (20 μ s)
Round trip trong cùng data center	50,000 ns (50 μ s)
Đọc tuần tự 1MB từ RAM	64,000 ns (64 μ s)

Hoạt động	Latency
Đọc 1MB qua mạng 100 Gbps	100,000 ns (100 μ s)
Đọc 1MB từ SSD	1,000,000 ns (1 ms)
Tìm kiếm trên đĩa cứng (Disk seek HDD)	5,000,000 ns (5 ms)
Đọc tuần tự 1MB từ đĩa cứng (HDD)	10,000,000 ns (10 ms)
Gửi gói tin Cali -> Hà Lan -> Cali (Vòng quanh thế giới)	150,000,000 ns (150 ms)

4.1.1. Collocation

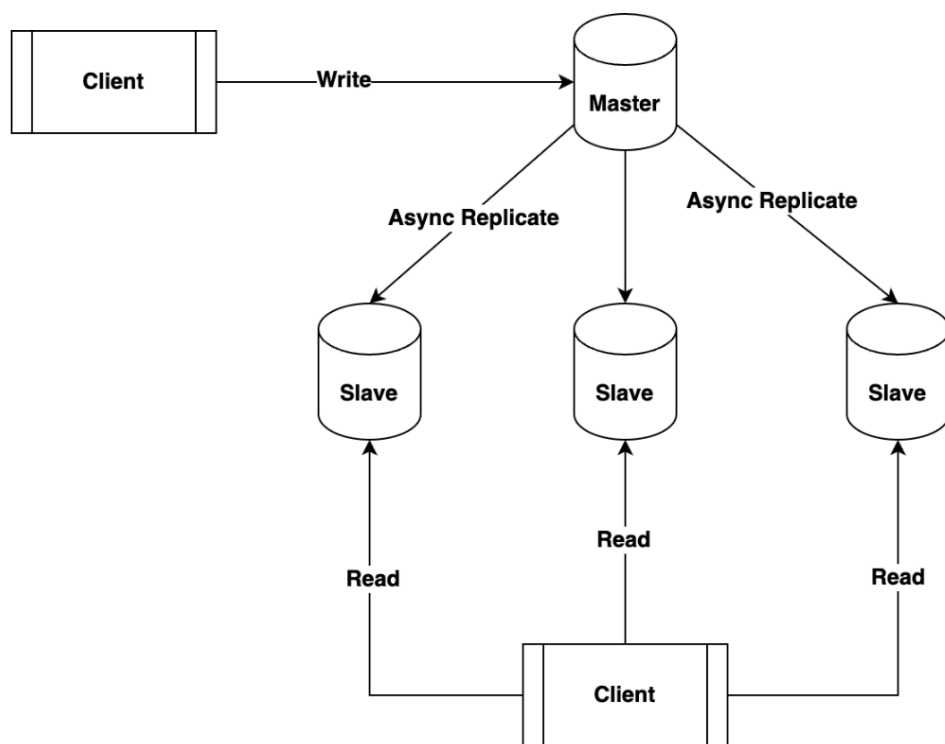
- Thay vì để App Server ở Việt Nam và Database Server ở Nhật Bản, đặt chúng trong cùng một Data Center, thậm chí cùng một rack.
- Trong kiến trúc Micro-services, nếu Service A gọi Service B rất nhiều, cần nhắc gộp chúng lại hoặc deploy chúng như các "sidecar" trên cùng một node (Kubernetes Pod) để giao tiếp qua localhost thay vì network.

4.1.2. Replication

Đưa dữ liệu đến gần người dùng hơn.

Trong System Design, một kỹ thuật phổ biến là **Master-Slave Architecture** (hay còn gọi là Leader-Follower).

- Mọi thao tác **INSERT**, **UPDATE**, **DELETE** bắt buộc phải gửi đến Master Database
- Master ghi dữ liệu vào đĩa, sau đó ghi log thay đổi (trong MySQL gọi là **binlog**, trong Postgres là **WAL file**)
- Các Slaves kết nối với Master, liên tục lắng nghe và copy log về để replay lại trên chính nó. Nhờ vậy, dữ liệu ở Slave giống hệt Master sau một khoảng trễ nhỏ.
- Các thao tác **SELECT** (Read) được điều hướng sang các Slaves.



Có hai cách mà Replication giúp hệ thống của bạn chạy nhanh hơn:

1. **Giảm Queuing Delay:** Đa số các ứng dụng web hiện nay là **Read-heavy** (Đọc nhiều, Ghi ít). Tỷ lệ thường là 90% Read - 10% Write (ví dụ: Facebook bạn lướt xem nhiều chứ đăng tuit thì ít). Nếu chỉ có 1 Database Server duy nhất, nó phải gánh cả Read lẫn Write. Khi traffic tăng, hàng đợi (Queue) vào CPU và Disk I/O của DB sẽ dài ra. Bằng cách thêm 5 Slave DB, bạn đã chia nhỏ lượng Read traffic ra làm 5 phần. Hàng đợi tại mỗi con Slave DB ngắn đi làm Latency giảm xuống.
2. **Giảm Network Delay:** Đặt các Slave DB rải rác gần user làm giảm đáng kể Network delay.

Tuy nhiên, không có bữa trưa nào là miễn phí. Khi chọn Replication để tối ưu Latency, bạn phải hy sinh Consistency (Tính nhất quán). Ví dụ, master node đặt tại Mỹ, slave node đặt tại Việt Nam sẽ mất 1 khoảng thời gian (gọi là Replication Lag từ vài trăm ms đến vài giây) để sync dữ liệu:

1. User A ở Việt Nam đăng bài mới lên Facebook. Request được gửi tới master ở Mỹ.
2. Ngay lập tức, User A F5 lại trang (request được gửi tới slave ở Việt Nam).
3. Do Replication Lag, Slave ở VN chưa kịp nhận data mới từ master.

4. User A hoang mang vì không thấy status mới của mình đâu.

Giải pháp cho vấn đề này là khi User vừa thực hiện một hành động cập nhật, thì trong vòng 1 khoảng thời gian (ví dụ 5s tiếp theo), trả các request Read của User đó vào Master (hoặc một Slave ưu tiên). Sau 1 phút, lại cho đọc từ Slave bình thường.

4.1.3. Local Caching

Local Caching là việc lưu dữ liệu ngay trong bộ nhớ RAM của chính process đang chạy ứng dụng (In-process memory). Với Local Caching, bạn chỉ đơn giản là truy cập vào một địa chỉ ô nhớ trong RAM hoặc CPU cache. Thời gian tốn là vài chục ns. Nó nhanh hơn Redis cả nghìn lần.

Tuy nhiên, Local Caching chỉ phù hợp với 2 loại dữ liệu:

- **Immutable Data (Dữ liệu bất biến):** Những thứ không bao giờ (hoặc cực hiếm khi) thay đổi. Ví dụ danh sách tỉnh thành, xã phường sau sáp nhập.
- **Tolerable Stale Data (Dữ liệu chấp nhận cũ một chút):** ví dụ số lượt like trên bài viết, dự báo thời tiết ngày mai.

Lưu ý

1. bạn nên dùng các thư viện chuyên dụng như Caffeine (Java) hay Ristretto/Go-cache (Go) cho Local Cache. Chúng cung cấp các tính năng mà HashMap bình thường không có như Eviction Policy, TTL (Time To Live), Size-based eviction (ví dụ cho phép cache tối đa 10000 items).
2. Set TTL ngắn và Max Size để tránh tràn RAM.

4.2. Avoid Work

Nếu bắt buộc phải làm gì đó, hãy làm nó ít nhất có thể.

4.2.1. Database Index

Nếu không có Index, để tìm user có `id=100`, Database phải quét từ dòng đầu tiên đến dòng cuối cùng của bảng (Full Table Scan). Độ phức tạp là $O(N)$. Nếu có Index (thường là cấu trúc B-Tree), Database đi theo nhánh cây với độ phức tạp là $O(\log N)$. Với bảng 1 triệu dòng:

- $N = 1.000.000$
- $\log(N) \approx 20$. Khác biệt là 50.000 lần!

Một vài lưu ý:

1. Over-indexing

Index giúp lệnh `SELECT` nhanh, nhưng làm lệnh `INSERT/UPDATE/DELETE` chậm đi. Vì mỗi khi bạn thêm dữ liệu, DB phải tốn công sắp xếp lại index. Vì vậy, chỉ nên index những cột hay dùng trong `WHERE`, `JOIN`, `ORDER BY`

2. Cardinality (Độ phân tán dữ liệu)

Nếu bảng có cột gender (Male/Female), bạn không nên đánh index cho nó vì Cardinality = 2 quá thấp. DB có thể phải lấy ra 50% dữ liệu của bảng, trong trường hợp này, Full Table Scan đôi khi còn nhanh hơn.

3. Composite Index: Bạn tạo một composite index trên 2 cột: (Lastname, Firstname).

- Query `WHERE Lastname = 'Hoang'` : Dùng Index.
- Query `WHERE Lastname = 'Hoang' AND Firstname = 'Quang'` : Dùng Index.
- Query `WHERE Firstname = 'Quang'` : KHÔNG dùng Index

4. Index Suppression: Nguyên tắc là không bao giờ bọc cột Index bên trong một hàm tính toán. Ví dụ bạn có cột `created_at` đã được đánh Index. Sếp yêu cầu: "Lấy cho anh tất cả đơn hàng của năm 2023". Sai lầm thường gặp

```
SELECT * FROM orders WHERE YEAR(created_at) = 2023;
```

Database lưu giá trị cụ thể (ví dụ `2023-05-15 10:30:00`) trong cây B-Tree. Nó không lưu giá trị `2023`. Khi bạn dùng hàm `YEAR()`, Database buộc phải đọc từng dòng trong bảng ra, chạy hàm `YEAR()` để xem kết quả có bằng 2023 không. Cách làm đúng:

```
SELECT * FROM orders
WHERE created_at >= '2023-01-01 00:00:00'
AND created_at <= '2023-12-31 23:59:59';
```

4.2.2. Optimize SQL Query

- Chỉ đọc những data cần thiết: không sử dụng `SELECT *` nếu không cần thiết.
- N+1 problems

Đây là lỗi kinh điển khi dùng ORM. Ví dụ, bạn muốn lấy danh sách 10 Users và Address của họ.


```
users = db.getUsers(); // 1 query
for (User u : users) {
    address = u.getAddress(); // 10 queries (mỗi vòng lặp 1 cái)
}
```

Tổng cộng 11 query. Thay vào đó chúng ta có thể dùng JOIN hoặc WHERE IN

```
SELECT * FROM addresses WHERE user_id IN (1, 2, ..., 10);
```

Chỉ tốn 2 query, latency giảm 10 lần!

- Ngoài ra còn rất nhiều kỹ thuật tối ưu SQL query, các bạn có thể tìm đọc chương 8 cuốn sách **High Performance MySQL**.

4.2.3. Optimize Code Logic

- Short-circuit: Đặt điều kiện nhẹ nhất hoặc dễ xảy ra nhất lên trước.

Ví dụ

```
if (isUserLoggedIn() && complexCalculation())
```

Nếu `isUserLoggedIn()` trả về False, hàm `complexCalculation()` tốn kém phía sau sẽ không bao giờ bị gọi.

- Tránh tạo Object thừa: ví dụ dùng StringBuilder thay vì cộng chuỗi String trong vòng lặp (vì String là immutable, mỗi lần cộng là tạo object mới)
- Chọn đúng cấu trúc dữ liệu: ví dụ Kiểm tra một phần tử có tồn tại trong danh sách không?
 - Dùng List: Độ phức tạp **O(N)**.
 - Dùng HashMap: Độ phức tạp **O(1)**.

4.2.4. Remote caching

Ở phần *Avoid Data Movement*, chúng ta nói về Caching như một cách để đem dữ liệu lại gần. Còn ở đây, hãy nhìn Caching như một cách **Avoid Work**.

Có những dữ liệu không có sẵn trong DB, mà phải tốn rất nhiều thời gian để tổng hợp. Ví dụ: "Top 10 sản phẩm bán chạy nhất tháng qua". Chiến thuật là chỉ tính 1 lần và lưu lại kết quả trong Remote Cache.

Remote Cache khác với **Local Cache** ở chỗ dữ liệu cache được lưu trong 1 server tập trung (ví dụ Redis/Memcached/DragonflyDB). Nhược điểm là sẽ tốn thêm network latency khi app server giao tiếp với cache server, nhưng Remote cache có hit rate cao hơn Local Cache.

Có nhiều pitfall (cạm bẫy) khi sử dụng cache nói chung và remote cache nói riêng:

- Thundering Herd
- Cache Invalidation
- Cache Penetration
- Cache Warmup
- Stale Set, Inconsistency.

Vì đây là một chủ đề rộng, xin phép trao đổi kỹ hơn với bạn đọc trong một cuốn ebook khác trong tương lai.

4.2.5. Thread Pool/ Connection Pool

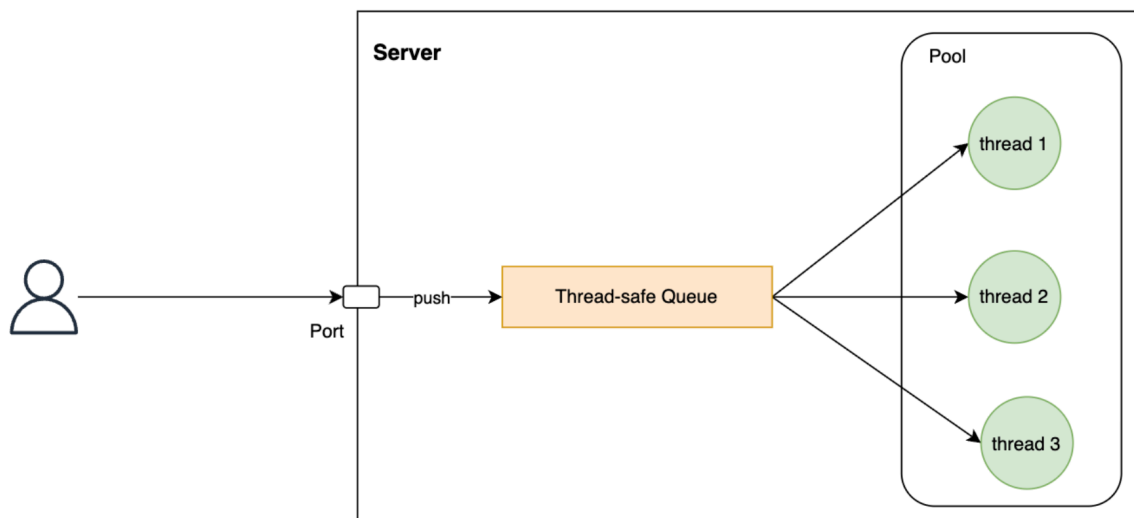
Tại sao chúng ta đi taxi mà không mua xe mới mỗi lần ra đường? Vì chi phí mua xe (setup cost) lớn hơn nhiều.

Để mở một kết nối tới DB (TCP/IP), cần:

1. Mở Socket
2. TCP 3-way handshake
3. SSL/TLS handshake
4. Authentication
5. Cấp phát Buffer phía Server

Nếu query của bạn chỉ mất 5ms, thì việc tạo connection tốn gấp 20 lần thời gian chạy query. Giải pháp là tạo sẵn 1 cái bể (Pool) chứa 50 connections. Dùng xong trả lại bể thay vì đóng lại.

Tương tự với thread, thay vì mỗi lần có request server tạo mới 1 thread để xử lý thì ta tạo sẵn 1 bể chứa thread. Các request tới server sẽ được cho vào 1 thread-safe queue, các thread trong pool sẽ lần lượt pull request từ queue này và xử lý.



Các bạn có thể tham khảo cách mình implement 1 TCP thread pool đơn giản bằng Golang [tại đây](#).

4.3. Avoid Waiting

Đừng chờ đợi!

Trong System Design, sự chờ đợi thường đến từ 2 phía:

- Chờ I/O (Disk/Network)
- Chờ tài nguyên (Locking).

4.3.1. Event Loop & I/O Multiplexing

- **Mô hình 1: Thread-per-Request**

Hãy tưởng tượng bạn sở hữu một nhà hàng.

- Bạn có 100 bàn.
- Bạn thuê 100 bồi bàn.
- Quy trình: Bồi bàn A ra bàn 1 nhận order → chạy vào bếp đưa phiếu → **đứng nhìn đầu bếp nấu** → nấu xong thì bưng ra.
- Trong lúc đứng nhìn đầu bếp (Waiting for I/O), bồi bàn A không làm gì cả, nhưng bạn vẫn phải trả lương.

Trong máy tính, Bồi bàn là **Thread**. Đầu bếp là **Database/Network**.

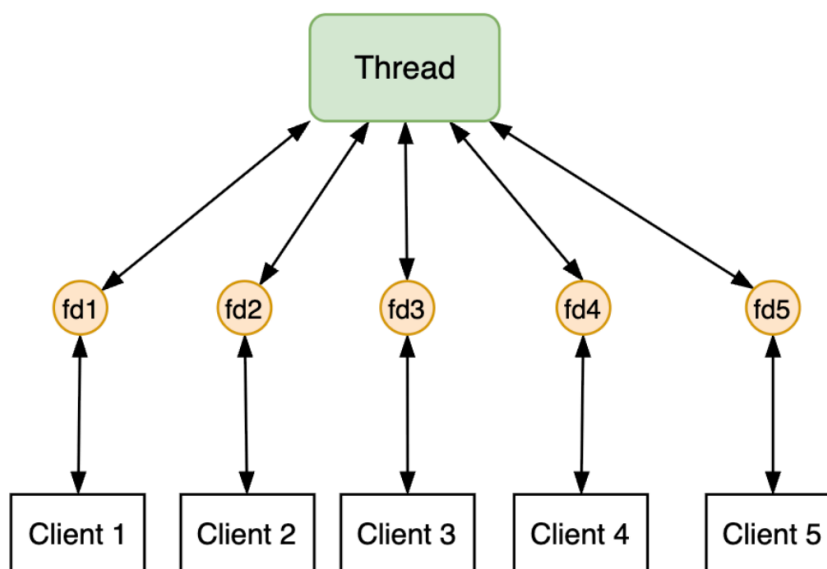
- **Mô hình 2: Event Loop**

Vấn nhà hàng đó, bạn chỉ thuê 1 bồi bàn.

- Quy trình: Bồi bàn nhận order bàn 1 → đưa phiếu vào bếp → **quay ngay ra** nhận order bàn 2 → đưa phiếu vào bếp → ...
- Khi bếp nấu xong món bàn 1, bếp trưởng bấm chuông "Ding!". Bồi bàn nghe chuông, quay lại bưng món ra bàn 1.

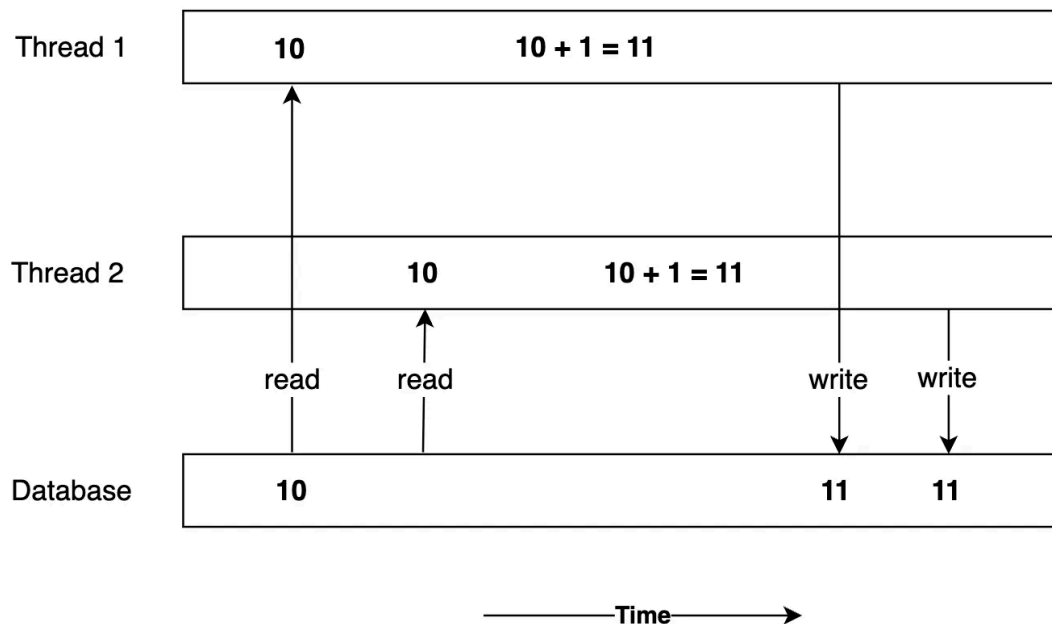
Công nghệ bên sau quy trình này là epoll/kqueue: làm sao 1 thread biết khi nào dữ liệu từ Network/Database đã sẵn sàng để xử lý? Nó không ngồi canh (poll) liên tục mà nhờ hệ điều hành (OS).

- Linux có tập hợp các system call đặc biệt tên là `epoll`. App sẽ sử dụng epoll và nói với OS: "Này OS, tôi đang quan tâm đến 1000 cái file descriptor này (mỗi file descriptor tương ứng với 1 user). Hãy theo dõi chúng, file descriptor nào có data về thì hãy thông báo cho tôi."
- Khi data đã sẵn sàng để xử lý, OS báo lại App, nhờ đó App có thể xử lý đồng thời nhiều request mà không bị block. Đây chính là cơ chế **I/O Multiplexing**. Nhờ cơ chế này, Nginx hay Redis có thể xử lý hàng chục nghìn kết nối chỉ với một vài threads.



4.3.2. Minimize Synchronization

Trong môi trường đa luồng (Multi-threading), để đảm bảo dữ liệu đúng, chúng ta dùng **Lock**. Trong ví dụ sau đây, nếu không dùng Lock kết quả là 11. Kết quả chính xác phải là 12:



quanghoang.substack.com

Cách làm đúng phải là

```

acquire_lock
  read(x)
  x = x + 1
  write(x)
release_lock
  
```

Tuy nhiên, sử dụng locking dẫn đến hiện tượng lock contention: các lock phải đợi nhau khi cùng muốn xử lý 1 data chung. Để giảm tranh chấp (Contention), bạn có 2 cách:

- **Lock Granularity:** Khóa phạm vi nhỏ thôi (ví dụ thay vì khóa cả bảng trong database thì khóa 1 dòng thôi).
- **Lock Duration:** Khóa nhanh rồi nhả ngay

Cùng xem xét một ví dụ đã được đơn giản hoá: Bạn viết hàm xử lý đơn hàng (Order Processing):

1. `BEGIN TRANSACTION`
2. `UPDATE products SET stock = stock - 1` (**Lock bắt đầu**).
3. `Write Log to File` (Tốn 10ms - Disk I/O).

4. `Call Payment API (Paypal)` (Tốn 2000ms - Network I/O).
5. `Send Email Confirmation` (Tốn 500ms).
6. `COMMIT` (**Lock kết thúc**).

Hậu quả là **dòng sản phẩm** đó bị khóa trong ~2.5 giây! Latency p90 sẽ tăng vọt.

Giải pháp:

1. `START TRANSACTION` → Trừ tiền kho, chuyển trạng thái thành Pending → `COMMIT`
2. `Call Payment API (Paypal)`
3. Nếu Paypal thành công → Update trạng thái đơn hàng "Paid". `Send Email Confirmation`
4. Nếu Paypal thất bại → Update trả lại tiền kho (Compensating Transaction).

✅ **Bài học:** không bao giờ thực hiện Network Call (HTTP, RPC) hoặc các tác vụ tốn thời gian (như đọc file, gửi email) bên trong một Database Transaction.

4.3.3. Partitioning / Sharding

Trong System Design, Partitioning không chỉ để chứa nhiều dữ liệu hơn, mà là để loại bỏ **Resource Contention (tranh chấp tài nguyên)**. (Lưu ý, mình sẽ dùng lẫn lộn Partitioning và Sharding, một vài sách sẽ phân biệt rạch ròi 2 khái niệm này, but not this one).

a. Database sharding

Hãy tưởng tượng Database của bạn là một cuốn sổ duy nhất. Dù bạn có server mạnh đến đâu, tại một thời điểm, ổ cứng (Disk Head) chỉ có thể ghi vào một chỗ.

Ví dụ khi bảng `Orders` của bạn lên tới 1 tỷ dòng.

- B-Tree Index trở nên khổng lồ (tăng I/O wait).
- Các transaction ghi dữ liệu bắt đầu chèn ép nhau (resource contention).

Giải pháp: Thay vì 1 Database to, ta chia thành N Database nhỏ (Shard), dựa trên User_ID.

- Users ID 0-1000 → Shard 1.
- Users ID 1001-2000 → Shard 2.
- ...

Khi User A (thuộc Shard 1) đặt hàng, DB chỉ lock một dòng trong Shard 1. Khi User B (thuộc Shard 10) đặt hàng, DB lock một dòng trong Shard 10. Hai hành động này

diễn ra trên 2 máy chủ vật lý khác nhau nên latency của User A không bị ảnh hưởng bởi dù User B đang spam transaction.

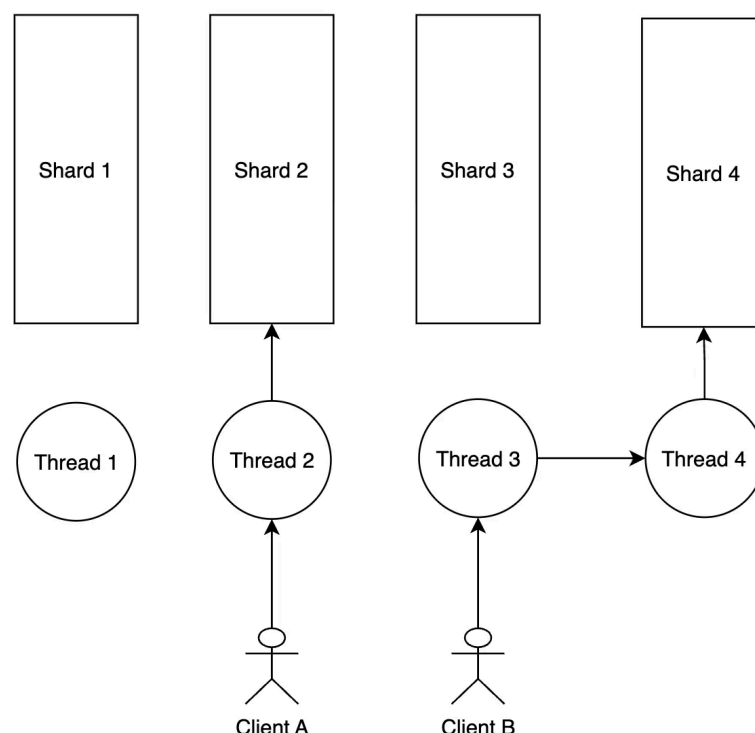
b. Case Study: DragonflyDB và kiến trúc **Shared-nothing architecture**

Đây là ví dụ hiện đại và thú vị về việc dùng Partitioning để giảm Latency xuống mức Micro-seconds.

Bối cảnh:

- **Redis:** Dùng kiến trúc Single-threaded.
 - *Ưu điểm:* Không cần Lock (vì chỉ có 1 thread).
 - *Nhược điểm:* "Waiting" theo kiểu xếp hàng. Request số 2 bắt buộc phải chờ Request số 1 làm xong. Không tận dụng được CPU nhiều cores.
- **Multi-threaded Key-Value Store (ví dụ Memcached):** Dùng nhiều threads.
 - *Ưu điểm:* Tận dụng nhiều cores.
 - *Nhược điểm:* "Waiting" do Lock. Khi 2 threads cùng muốn update một vùng nhớ, chúng phải chờ Mutex Lock.

Giải pháp của DragonflyDB: Partitioning ngay trong bộ nhớ RAM



Nếu máy chủ có 8 Core CPU, Dragonfly sẽ chia RAM thành 8 phần riêng biệt (8 Shard).

- Mỗi Core CPU sẽ quản lý tuyệt đối một shard.
 - Core 1 chỉ xử lý các keys hash vào slot 1.
 - Core 2 chỉ xử lý các keys hash vào slot 2.

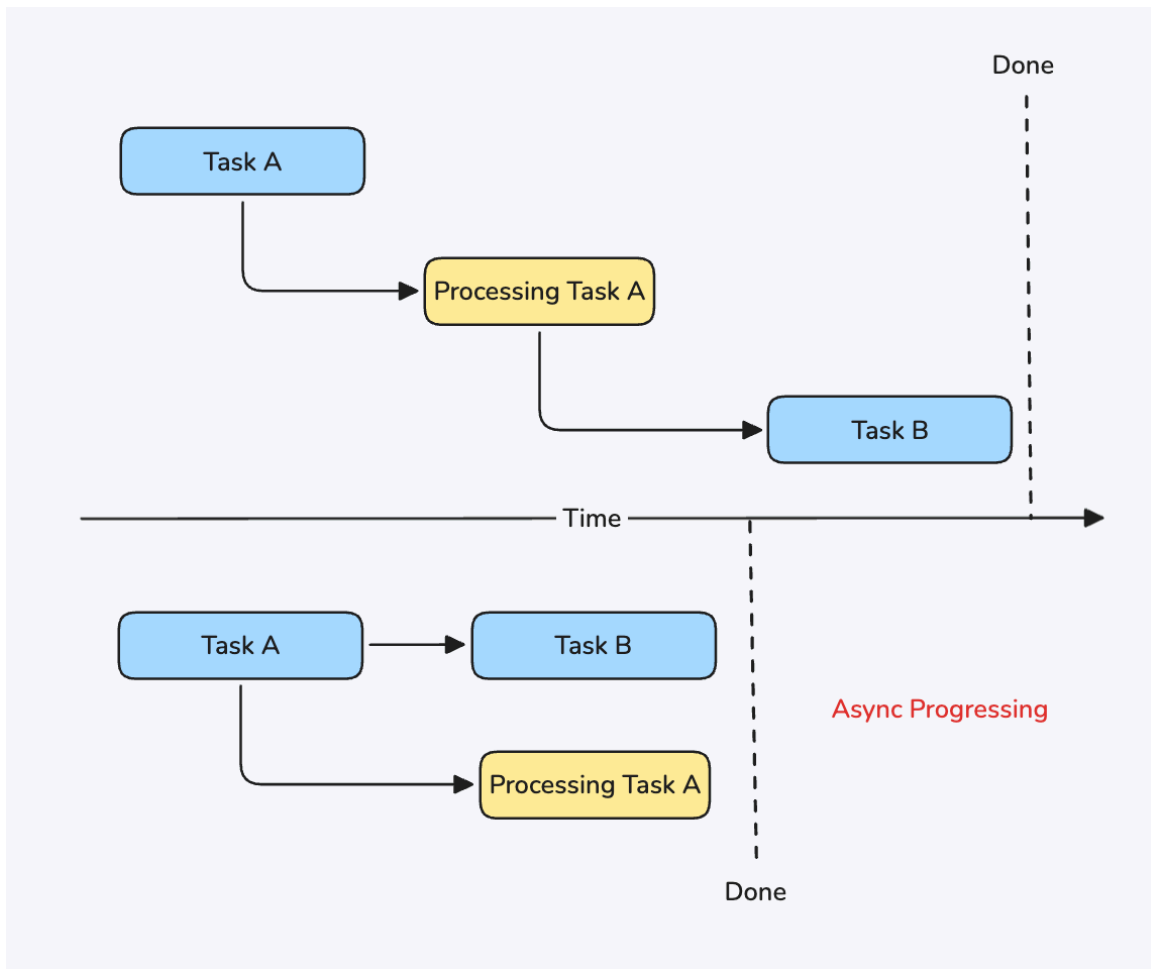
Vì Core 1 biết chắc chắn rằng không core nào khác sẽ động đến shard 1, nên Core 1 không cần dùng Lock. Ý tưởng này gọi là **Shared-nothing** (Không chia sẻ gì cả): Không chia sẻ bộ nhớ → Không cần Lock → Không có Waiting.

Kết quả benchmark thực tế cho thấy DragonflyDB có thể đạt hàng triệu ops/sec với Latency thấp hơn Redis tới 25 lần trên các instance lớn (nguồn), đơn giản vì nó đã triệt tiêu được thời gian chờ đợi do xếp hàng (Redis) hoặc do lock contention (Memcached).

4.4. Hide Latency

Đôi khi chúng ta không thể làm hệ thống chạy nhanh hơn được nữa (do giới hạn vật lý hoặc logic quá phức tạp). **Hiding Latency** không phải là làm cho **T** nhỏ lại, mà là làm cho **T** trở nên vô nghĩa hoặc ít gây khó chịu hơn

4.4.1. Async Processing



Ví dụ: user upload một video 1GB lên Youtube. Server cần: Nhận file → Scan virus → Check tiêu chuẩn cộng đồng, bản quyền → Convert định dạng (Transcode) → Lưu vào CDN → ...

Quy trình này mất 10 phút nhưng Youtube sẽ trả về ngay “Upload thành công! Chúng tôi đang xử lý.” Đây chính là một ví dụ của **Async Processing**. Youtube đã tách (decouple) workload làm 2 phần:

- Ingest (nhận yêu cầu)
- Process (xử lý yêu cầu)

4.4.2. Parallelism

Một trong những cách phổ biến nhất để "giấu" latency là **Parallelism** (Xử lý song song).

Ví dụ: Để load User Profile, bạn cần gọi 3 dịch vụ:

1. Lấy thông tin cơ bản (User Info): 50ms
2. Lấy danh sách bạn bè (Friends): 100ms

3. Lấy lịch sử đơn hàng (Orders): 150ms

Nếu chạy tuần tự (Sequential): Tổng = 300ms. Nếu chạy song song (Parallel): Tổng = Max(50, 100, 150) = 150ms. Bạn vừa giảm được 50% latency!

Nhưng bạn có thể giảm xuống 1ms bằng cách thêm 1000 servers chạy song song không?

Gene Amdahl bảo là “**Không**”. Định luật Amdahl phát biểu rằng:

Tốc độ tối đa của một chương trình bị giới hạn bởi phần **tuần tự** (serial part) của nó.

Hãy nhìn vào công thức tính hệ số tăng tốc:

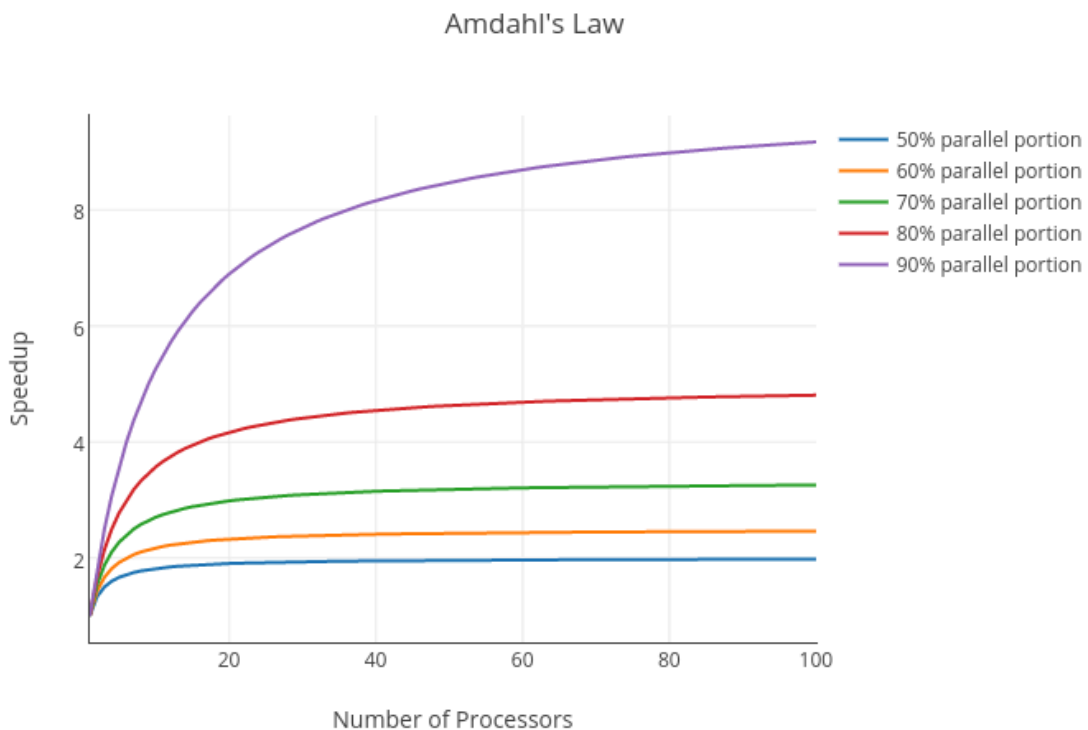
$$Speedup = \frac{1}{(1 - P) + \frac{P}{N}}$$

- **P:** Tỷ lệ phần công việc có thể chạy song song (ví dụ 90% code của bạn chạy song song được, P = 0.9).
- **1 - P:** Phần bắt buộc phải chạy tuần tự (ví dụ 10% code là khởi tạo variable, đợi lock, gửi kết quả về, đây là Serial part).
- **N:** Số lượng processors (số nhân CPU hoặc số server).

Nếu chương trình của bạn có **50%** là chạy tuần tự (P = 50%), thì dù bạn có ném vào đó bao nhiêu CPU đi nữa, hệ thống cũng chỉ nhanh lên tối đa là **2 lần** (1/0.5).

Bài học:

- **Optimise Serial Part First:** Nếu muốn hệ thống nhanh hơn, đừng vội mua thêm server. Hãy tối ưu vào những phần "nút cổ chai" (bottleneck) trước.
- **Diminishing Returns** (Hiệu suất giảm dần): Nhìn vào biểu đồ Amdahl, bạn sẽ thấy đường cong đi ngang rất nhanh. Giả sử P=1, ban đầu thêm 2 server thì nhanh gấp đôi, nhưng khi lên đến 100 server, việc thêm server thứ 101 gần như không tác dụng gì.



4.4.3. Request Hedging

Đây là kỹ thuật cao cấp, được Google phổ biến qua bài báo *The Tail at Scale*. Ý tưởng của Hedged request rất đơn giản: gửi cùng 1 request tới nhiều replica server, và sử dụng kết quả từ replica server phản hồi sớm nhất. Cụ thể:

- đầu tiên, gửi 1 request tới replica server **X** và đợi một khoảng thời gian **D (ms)**, nếu **X** vẫn chưa phản hồi thì tiếp tục gửi 1 request (hedged request) tới replica server **Y** khác.
- sau khi nhận được phản hồi đầu tiên từ **X** hoặc **Y**, hủy request còn lại.
- lặp lại quá trình này nếu cả **X** và **Y** đều không phản hồi sau khoảng thời gian $2 * D$.

Có 2 lưu ý quan trọng:

1. API cung cấp bởi các replica server phải có tính idempotent, nghĩa là cùng một request được gửi đi gửi lại nhiều lần sẽ luôn cho ra cùng một kết quả ban đầu. Ví dụ bạn gửi request để thanh toán cùng một order 10 lần, thẻ tín dụng của bạn chỉ bị trừ tiền một lần.
2. Phải chọn khoảng thời gian chờ **D** một cách phù hợp.

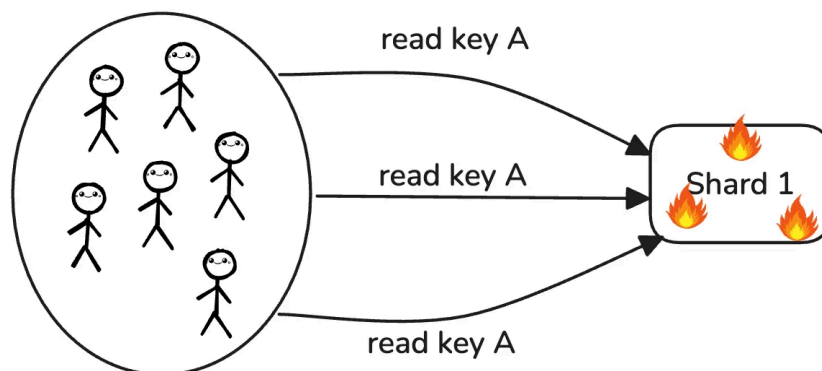
- Nếu D quá nhỏ sẽ dẫn đến overload hệ thống, khi mà quá nhiều hedged request được gửi về dẫn đến hiện tượng Retry Storm.
- Nếu D quá lớn sẽ làm giảm hiệu quả của Hedge request.

Trên thực tế, Google đã từng gặp sự cố liên quan đến việc chọn giá trị thời gian chờ D quá nhỏ lẫn quá lớn. Giải pháp là chọn giá trị D một cách linh hoạt và bằng với giá trị p95 latency hiện tại. Nói cách khác, ta chỉ gửi hedged request cho 5% request chậm nhất. Điều này đảm bảo lượng tải phát sinh từ hedged request được giới hạn trong khoảng 5% tổng số request, trong khi vẫn đảm bảo hiệu quả giảm thiểu long-tail latency.

Thử nghiệm thực tế cho thấy, khi đọc 1000 key từ 100 server Google Bigtable với D = 10ms, **p99.9 latency đã giảm từ 1800ms xuống còn 74ms trong khi chỉ tăng load cho hệ thống 2%.**

4.4.4. Request Collapsing (Singleflight)

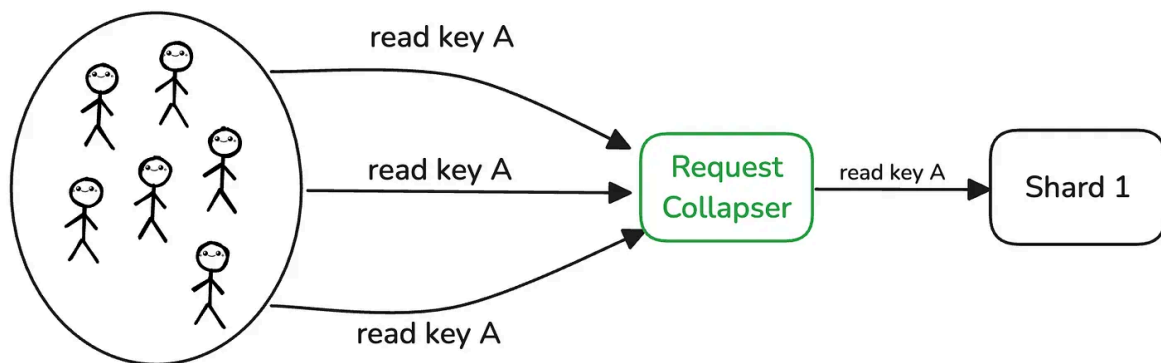
Kỹ thuật này để trị vấn đề **Thundering Herd**. Ví dụ: 1000 users cùng truy cập trang chủ vào đúng 12:00:00. Tất cả đều hỏi Cache: "Cho tôi lấy key A". Giả sử lúc đó Cache vừa hết hạn (Miss). 1000 requests này sẽ cùng lúc lao thẳng vào Database (Cache Stampede). Database bị quá tải hoặc tệt hơn là sập. Latency tăng vọt.



Giải pháp là sử dụng pattern Singleflight

1. Nhận request 1: "Lấy key A". → Cache miss → Query DB
2. Trong lúc đang gọi DB (mất 10ms), server nhận tiếp request 2, 3... đến 1000.
3. Server **giữ** 999 request này lại, không gọi DB nữa. Chúng sẽ chờ kết quả của request 1.
4. Khi request 1 có kết quả từ DB, Server trả kết quả đó cho cả 1000 request.

Kết quả: Database chỉ chịu 1 query thay vì 1000. Latency của request thứ 1000 chỉ bằng Latency của request đầu tiên.



4.4.5. Speculative Prefetching

Định cao của Hide Latency là làm trước khi user kịp yêu cầu. Ví dụ:

1. User đang đọc trang 1. 90% họ sẽ bấm sang trang 2. Trong lúc họ đang đọc, frontend ngầm gửi request tải trang 2 về. Khi User bấm "Next", trang 2 hiện ra tức thì.
2. Trong form đăng ký, khi User điền xong field "Username" và đang chuyển sang field "Password", hệ thống đã có thể ngầm gửi request check "Username này có bị trùng không?". Đến khi User bấm Submit, kết quả check đã có sẵn.

Lời Kết

Trong System Design, không có giải pháp nào là hoàn hảo, chỉ có sự đánh đổi. Mọi nỗ lực giảm Latency đều đi kèm với một cái giá phải trả:

- Complexity (Sự phức tạp)
- Consistency (Tính nhất quán)
- Cost (Chi phí)

“We should forget about small efficiencies, say about 97% of the time: **premature optimization is the root of all evil**”

— Donald Knuth



Hãy bắt đầu bằng việc **Đo đạc**

- Hãy nhìn vào biểu đồ P95, P99.
- Hãy tìm ra đúng điểm nghẽn (Bottleneck). Khắc phục nó.
- Và biết điểm dừng (Good enough).

Chúc các bạn xây dựng được những hệ thống không chỉ nhanh, mà còn bền vững.

Tài liệu tham khảo thêm

1. [Performance Hints](#) by Jeff Dean, Sanjay Ghemawat
2. [Performance tips of the week](#)
3. [Latency](#) by Pekka Enberg
4. [High Performance MySQL](#) by Silvia Botros, Jeremy Tinley
5. [P99 Conf](#)

Về tác giả



Chào bạn, mình là **Quang Hoàng**.

Tại thời điểm viết cuốn sách này, mình đang là Software Engineer tại Google. Trước khi gia nhập Google, mình đã từng có thời gian làm việc tại Shopee và Rakuten – những môi trường mà khái niệm High Concurrency và Low Latency không chỉ là lý thuyết, mà là vấn đề sống còn của hệ thống.

Nếu bạn muốn trao đổi thêm về System Design, hoặc đơn giản là kết nối với một người cùng đam mê, hãy ghé thăm mình tại:

- **Blog:** <https://quanghoang.substack.com/>
- **LinkedIn:** <https://www.linkedin.com/in/hoangdquang/>